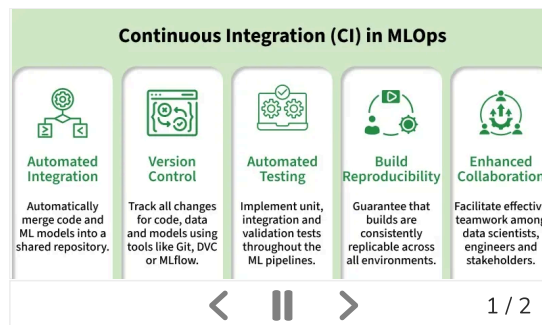


Continuous Integration and Continuous Deployment (CI/CD) in MLOps

Last Updated : 22 Nov, 2025

In MLOps, Continuous Integration (CI) and Continuous Deployment (CD) help automate the development, testing and deployment of machine learning models. Adapting these practices from software engineering makes ML pipelines more reliable, consistent and easier to scale.



Why CI/CD is Needed in MLOps

Unlike traditional software, ML models depend on:

- Code
- Data
- Features
- Model artifacts
- Infrastructure

Because these components change frequently, CI/CD pipelines help maintain reproducibility, quality and speed throughout the model lifecycle. Every update whether it's new data, a changed hyperparameter or a bug fix must go through a structured pipeline to avoid unpredictable behavior in production.

Continuous Integration (CI) in MLOps

CI in MLOps focuses on validating changes across multiple ML components.

1. **Data Validation:** Checks for schema changes, missing values, drift and anomalies.
2. **Feature Validation:** Ensures feature transformations remain consistent and correct.
3. **Model Training Automation:** Re-trains models automatically when new code/data is pushed.
4. **Unit & Integration Tests:** Tests for preprocessing pipelines, scripts and model behavior.
5. **Model Evaluation Tests:** Accuracy, precision, recall, ROC-AUC, fairness, explainability checks.

Outcome of CI: A validated ML model artifact stored in a model registry ready for deployment.

Continuous Deployment (CD) in MLOps

CD automates the release of validated ML models to production with safety checks. Key Components of CD in MLOps

1. **Model Registry Integration:** Automatically selects the newest validated model version.
2. **Deployment Strategies:**
 - **Blue-Green Deployment:** Two identical environments; switch traffic only when stable.
 - **Canary Release:** Release the model to a small portion of users first.
 - **Shadow Deployment:** New model runs parallelly without affecting real users.
3. **Automated Performance Monitoring:**
 - Model drift
 - Data drift
 - Latency and throughput
 - Real-time accuracy (where available)
4. **Rollback Mechanisms:** Automatically revert to the previous model if performance drops.

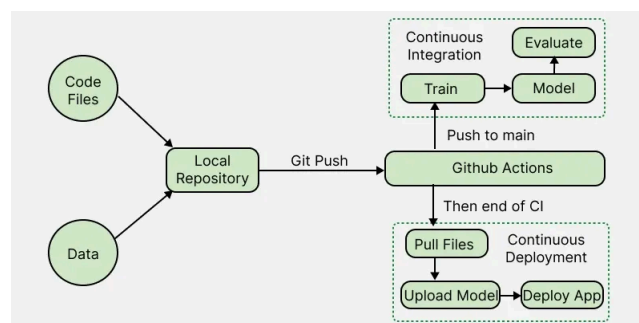
How CI/CD Improves the ML

Lifecycle

- **Efficiency:** Automates repetitive steps like training, testing, packaging and deployment.
- **Reliability:** Prevents bad models from reaching production by enforcing checks.
- **Scalability:** Makes it easy to manage multiple models across teams and environments.
- **Reproducibility:** Tracks every version of data, code and model used in training.
- **Faster Experimentation:** Data scientists can focus on modeling rather than DevOps work.

Example Project (Full CICD Workflow)

To understand CI/CD in [MLOps](#), consider this simple project where we automate the entire ML workflow from training to deployment using scikit-learn, GitHub Actions, CML and Hugging Face Spaces.



Example workflow

We build a Random Forest drug classifier using scikit-learn pipelines. Every time we push new code or data to GitHub:

1. The model is automatically retrained.
2. Metrics and confusion matrix are generated using CML.
3. Updated model + results are saved.
4. The web app and model are redeployed to Hugging Face Space automatically.

This creates a fully automated CI/CD ML pipeline.

Steps

1. Project Structure

App/ -> Gradio app
Data/ -> drug.csv

Model/ -> Saved pipeline
Results/ -> metrics + plots
train.py -> Training script
Makefile -> Commands for CI/CD
ci.yml -> CI workflow
cd.yml -> CD workflow

2. Training Script (train.py)

1. Load CSV (drug.csv)
2. Split into train/test
3. Build preprocessing + RandomForest pipeline
4. Evaluate accuracy and F1
5. Save:
 - metrics -> Results/metrics.txt
 - confusion matrix -> Results/model_results.png
 - model -> Model/drug_pipeline.skops

3. CI Pipeline (GitHub Actions + CML)

The CI workflow performs:

1. Install dependencies
2. Format code
3. Train the model (run train.py)
4. Generate metrics + confusion matrix
5. Create a CML report under the GitHub commit

Triggered automatically on every push.

ci.yml calls Makefile commands:

```
make install
make train
make eval
```

4. CD Pipeline (Deploy to Hugging Face)

Once CI finishes successfully:

1. CD workflow pulls updated model/results
2. Logs in using Hugging Face token
3. Uploads:
 - App folder (Gradio app)
 - Model folder
 - Results folder
4. Automatically updates the Hugging Face Space

Triggered by:

```
on:  
  workflow_run:  
    workflows: ["Continuous Integration"]  
    types: [completed]
```

5. Gradio App (drug_app.py)

- Loads saved scikit-learn pipeline
- Accepts user inputs (age, sex, cholesterol, BP, Na/K ratio)
- Predicts the drug type
- Runs on Hugging Face Space with continuous deployment

Short Outcome

With CI/CD:

- Push code -> Model retrains
- Metrics are auto-generated
- New model is deployed to Hugging Face within minutes
- No manual steps needed

This is a compact, production-ready example of CI/CD in MLOps using real tools.

Challenges in Implementing CI/CD for ML

- **Data dependencies:** Data changes frequently and must be validated continuously.
- **Complexity of ML pipelines:** Feature engineering, model training and evaluation add layers of complexity.
- **Compute requirements:** Training pipelines may require GPUs or distributed computing.
- **Monitoring difficulties:** Real-world data drift makes monitoring an important and difficult task.
- **Cross-functional coordination:** Data scientists, DevOps and ML engineers must collaborate closely.

Best Practices for CI/CD in MLOps

- Use a model registry (MLflow, Vertex AI Registry, SageMaker Registry)
- Implement automated data/feature validation
- Use containerized environments with Docker

- Automate end-to-end pipelines using tools like: GitHub Actions, Jenkins, GitLab CI, Kubeflow, Airflow
- Use tracking systems for experiments (MLflow, Weights & Biases)
- Monitor data and model drift continuously
- Practice safe deployment strategies (canary/shadow release)

Suggested Quiz

↺ 6 Questions

What does CI/CD stand for in MLOps?

- ☐ A Continuous Integration/Continuous Delivery
- ☐ B Code Integration/Code Deployment
- ☐ C Cluster Initialization/Code Delivery
- ☐ D Continuous Improvement/Continuous Development

[Login to View
Explanation](#)

1/6 < Previous
Next >

Comment

K ksri3rlry

1

Article Tags: [Machine Learning](#)

[Machine Learning Blogs](#)

Explore

Machine Learning Basics

Python for Machine Learning

Feature Engineering

Supervised Learning

Unsupervised Learning

Model Evaluation and Tuning

Advanced Techniques

Machine Learning Practice

Courses



Corporate & Communications Address:

A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)



Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305



Company

About Us
Legal
Privacy
Policy
Contact Us
Advertise
with us
GFG
Corporate
Solution

Training
Program

Explore

POTD
Job-A-
Thon
Blogs
Nation
Skill Up

Tutorials

Programming
Languages
DSA
Web
Technology
AI, ML &
Data Science
DevOps
CS Core
Subjects
Interview
Preparation
Software and
Tools

Courses

ML and Data
Science
DSA and
Placements
Web
Development
Programming
Languages
DevOps &
Cloud
GATE
Trending
Technologies

Videos

DSA
Python
Java
C++
Web
Development
Data Science
CS Subjects

Preparation Corner

Interview
Corner
Aptitude
Puzzles
GfG 160
System Design